COP2034C Module 11 Programming Assignment

Instructions

In this assignment, we will continue developing our Palmer Penguins program from Module 10. Previously, we introduced exception handling to manage invalid input, missing observations, and problems accessing CSV data.

In Module 11, we will focus on organizing Python programs using modules. We already introduced a custom module, penguin.py, in Module 9. Now we will extend that concept by moving several of our existing statistical functions into a separate module and importing them into our main program.

We will practice selected Chapter 11 concepts:

- Creating and importing custom Python modules.
- Accessing module functions using dot notation.
- Importing specific functions using from and import.
- Using the if __name__ == "__main__": statement to control program execution.

We will retain our existing Palmer Penguins functionality while improving the organization and reusability of the code.

1. Start with Your Module 10 Solution

Open your completed Module 10 notebook in Google Colab and make a copy for Module 11. Suggested notebook name:

```
PalmerPenguinsM11.ipynb
```

The downloaded Python program should be named:

```
PalmerPenguinsM11.py
```

Update the ID header:

```
# PalmerPenguinsM11.py
# Your Name
# Submission Date
# Organize Palmer Penguin analysis using custom modules
```

Retain the working functionality from Module 10, including species statistics, CSV processing, exception handling, body mass summaries, dictionary operations, Penguin objects, and interactive species exploration.

NOTE: Your existing penguin.py file does not require modification.

2. Create a Custom Statistics Module

Our main program currently contains several functions that perform calculations or classify statistical results. These functions could be useful in other programs that analyze different datasets. We will move three existing functions into a new custom module named

```
penguin_stats.py
```

Create a new code cell in Google Colab and enter:

```
%%writefile penguin_stats.py
```

This Colab command writes the code in the cell to a Python file named penguin_stats.py. Move the following functions from your main program into this new module:

```
calculate_percentage(count, total)

classify_species(percent)

get_position_description(selected_species, largest_species,
                         smallest_species)
```

Keep their existing implementations unchanged.

At the beginning of penguin_stats.py, include an ID header:

```
# penguin_stats.py
# Your Name
# Submission Date
# Functions for calculating and classifying penguin statistics
```

Include a short descriptive comment above each function explaining its purpose. Remember not to include the %%writefile command in the downloaded .py file; it is a Google Colab command, not standard Python syntax. Once you have created the new module, remove the three function definitions from the code cell containing your main solution (PalmerPenguinsM11.py). The functions should now be defined only in penguin_stats.py.

3. Import and Use the Statistics Module

We will now use the functions from our new module in the main program. At the beginning of PalmerPenguinsM11.py, add:

```
import penguin_stats
```

Retain the existing imports:

```
import math
from penguin import Penguin
```

Notice that we are using two different import approaches.

"import penguin_stats" imports the module, allowing us to access its functions using the module name followed by dot notation. For example:

```
percentage = penguin_stats.calculate_percentage(
                    species_counts[0], total_penguins)
```

"from penguin import Penguin" imports the Penguin class directly, allowing us to create objects without prefixing the class name with the module name.

Update the existing function calls

Locate all calls to the three functions moved into penguin_stats.py. Update each call to include the module name.

Existing call	**Updated call**
calculate_percentage(...)	penguin_stats.calculate_percentage(...)
classify_species(...)	penguin_stats.classify_species(...)
get_position_description(...)	penguin_stats.get_position_description(...)

For example, the existing calculation

```
chinstrap_percent = calculate_percentage(
     species_counts[0], total_penguins)
```

should become

```
chinstrap_percent = penguin_stats.calculate_percentage(
     species_counts[0], total_penguins)
```

Make the corresponding changes to the remaining calls. Do not change the calculations performed by these functions. Run the program and verify that the species percentages and interactive exploration still produce the same results as Module 10.

4. Demonstrate Importing a Specific Function

Python also allows us to import individual functions directly from a module. We will demonstrate this approach without changing the existing calls that use dot notation.

At the beginning of the main program, add:

```
from penguin_stats import calculate_percentage
```

Both import statements should now be present:

```
import penguin_stats
from penguin_stats import calculate_percentage
```

Inside main(), immediately after the existing species statistics and average calculations, add:

```
print("\nModule Import Demonstration")
percentage = calculate_percentage(
    species_counts[0], total_penguins)
print(f"Chinstrap percentage: {percentage:.2f}%")
```

This demonstrates use of the directly imported function instead of calling it through the module name. Expected output:

```
Module Import Demonstration
Chinstrap percentage: 19.88%
```

The original species percentage calculations should continue using penguin_stats.calculate_percentage(). This allows us to demonstrate both import approaches within the same program.

5. Display the Location of an Imported Module

When Python imports a module, it searches for the module in several locations, including the current working directory and directories listed in Python's module search path. We can inspect the location of a successfully imported module using its **__file__** attribute. Inside main(), immediately after the Module Import Demonstration, add:

```
print("\nCustom Module Location")
print(penguin_stats.__file__)
```

The output should display the path to penguin_stats.py. For example, in Google Colab, the output will usually look like this:

```
Custom Module Location
/content/penguin_stats.py
```

The exact path may differ depending on the environment. This demonstrates that Python has located and imported the module file.

6. Use the Main Module Guard

Chapter 11 explains that a Python file can be executed as a script or imported as a module. We learned in a previous module that when a Python file is executed directly, the special variable **__name__** contains the value **"__main__"**. When the file is imported by another program, **__name__** will usually contain the module's name.

Currently, your Palmer Penguins program may end with:

```
main()
```

If you have not done so already, replace this statement with:

```
if __name__ == "__main__":
    main()
```

This ensures that main() executes when we run PalmerPenguinsM11.py directly. But if another Python program imports PalmerPenguinsM11, the interactive program will not automatically execute. This is important because our program contains input prompts; we don't want those prompts to appear simply because another program imports our module.

The main module guard should not change the output when the program is executed normally.

7. Organize the Updated Program

The Module 11 changes should be integrated into the existing program.

Your solution should now contain three Python files:

Filename	Purpose
PalmerPenguinsM11.py	Main program containing the analysis and interactive exploration
penguin.py	Existing custom module containing the Penguin class
penguin_stats.py	New custom module containing three statistical functions

The main program should follow this general structure:

```
PalmerPenguinsM11.py
    import required modules
    define species constants
    define remaining program functions
    main()
        display species statistics
        demonstrate direct function import
        display custom module location
        process sample observations
        demonstrate exception handling
        calculate body mass statistics
        display species dictionary
        create and display Penguin objects
        calculate object-based statistics
        explore species interactively
        display final exploration summary

    if __name__ == "__main__":
        main()
```

The three functions moved into penguin_stats.py should no longer be defined in the main program. All existing functionality should continue working.

Expected Output

> **NOTE: Before executing your main program, execute the cell that creates penguin_stats.py. If you modify the module after it has already been imported, you may need to restart the notebook runtime or reload the module to see the changes.**

Your program should retain the existing Module 10 output, including species statistics, CSV observations, exception handling, body mass summaries, Penguin objects, and interactive species exploration.

The new output should include:

```
Palmer Penguins Dataset

Chinstrap: 68 (19.88%)
Gentoo: 123 (35.96%)
Adelie: 151 (44.15%)

Total penguins: 342
Average number of penguins per species: 114.00
Average rounded up: 114

Module Import Demonstration
Chinstrap percentage: 19.88%

Custom Module Location
/content/penguin_stats.py
```

The remainder of the program should continue producing the same results as Module 10. For example, the existing exception-handling demonstration should still display:

```
Exception Handling Demonstration
Error: Penguin observation 999 could not be retrieved.
```

The interactive exploration should continue supporting Chinstrap, Gentoo, Adelie, Sample, and Done. The new module organization should not change the statistical results or existing program behavior. Minor whitespace differences and differences in the displayed module path are acceptable.

Test Your Program

- Before submitting, verify that:
- The new penguin_stats.py module contains the three required functions.
- The three functions have been removed from the main program.
- The main program imports penguin_stats successfully.
- The existing function calls use the module name and dot notation.
- The direct function import demonstration displays 19.88%.
- The custom module location is displayed.
- The main module guard is present and correctly indented.
- The existing penguin.py module continues working.
- The exception-handling demonstrations from Module 10 still work.
- The interactive species exploration continues working.
- Entering Done ends the interactive loop.
- The program produces no unexpected errors.

Data Science Connection

In previous modules, we developed a program that reads Palmer Penguins observations, calculates statistical summaries, creates objects, and handles errors during data processing. As data science projects become more complex, placing all of the code in a single file can make a program difficult to understand, test, and maintain. Modules provide a way to organize related functionality into separate Python files. For example, a data science project might use different modules for:

- **Data retrieval:** Reading observations from CSV files or databases.
- **Data preparation:** Converting values and handling missing data.
- **Statistical analysis:** Calculating percentages, averages, and other summary statistics.
- **Data visualization:** Creating charts and graphs.
- **Model evaluation:** Measuring the performance of machine learning models.

In our Palmer Penguins project, penguin_stats.py now contains reusable statistical functions that are separate from the main program. This organization offers several advantages.

- **Code reuse:** The same statistical functions can be imported into multiple programs without rewriting them.
- **Maintainability:** Changes to a calculation can be made in one module without modifying every program that uses it.
- **Organization:** Related functions are grouped together, making the project easier to navigate.
- **Testing:** Individual functions can be tested separately from the interactive program.

Python's data science ecosystem relies extensively on modules and packages. Libraries like NumPy, pandas, and Matplotlib organize their functionality into modules that programmers import as needed (we will look at these libraries in an upcoming course module).

Our Palmer Penguins program now demonstrates this same organizational principle on a smaller scale.

Readability

Continue following the coding conventions established in previous modules:

- Use meaningful variable and function names.
- Use consistent four-space indentation.
- Include descriptive comments above important functions and operations.
- Use logical blank lines to separate sections.
- Place import statements near the beginning of each Python file.
- Avoid duplicating function definitions across modules.
- Use module names that clearly describe their purpose.
- Preserve the existing organization and functionality of the program.

The new module should make the program easier to understand without unnecessarily changing working code.

Screenshot and Submission

Run the completed program in Google Colab. Capture enough of the execution output to demonstrate the new module import functionality, the custom module location, and the existing Palmer Penguins analysis.

Download the following files from Google Colab:

Filename	Description
PalmerPenguinsM11.py	Main Python program using the imported statistical functions
penguin.py	Existing custom module containing the Penguin class
penguin_stats.py	New custom module containing the three statistical functions
PalmerPenguinsM11.png	Screenshot of the program's execution output

To download the main Python program, use the appropriate Google Colab download option. Make sure the %%writefile commands are not included in the exported program, since they are Colab-specific and are not valid Python syntax.

To download the custom modules, open the Files panel on the left side of Colab. Locate each .py file, click the three-dot menu next to its filename, and select Download.

Upload all four files to your forked GitHub repository and submit the repository URL to Canvas. Do not submit the .ipynb notebook.